SYMANEK COLLEGE

Guia Técnico de Operação, Acessos e Produção

Documento técnico — Supabase, Vercel, permissões, validação, riscos e manutenção

Projeto	**Symanek College** — Site público (Next.js) + Suite de gestão (Vite/React) + Supabase
Documento	Guia Técnico de Operação, Acessos e Produção
Versão	1.0
Data	29 de agosto de 2026
Responsável técnico / aceite	Pedro Teixeira — pedroteixeira201435@gmail.com

Segurança: Este documento **não contém** tokens, passwords, service role keys nem segredos. Nunca colar segredos aqui. Onde forem necessárias credenciais usar marcadores: `[email do admin]`, `[email do estudante]`, `[password temporária]`.

Índice

1. Visão geral da arquitetura
2. Links e ambientes
3. Variáveis de ambiente
4. Como criar acessos reais no Supabase
5. Como dar acesso a um estudante
6. Como rever dados antes de produção
7. Capacidades dos cursos
8. Segurança / RLS / RPC
9. Auditoria
10. CI/CD

11. Deploy na Vercel
12. Comandos úteis
13. Rotação de segredos
14. Backup e recuperação
15. Critérios técnicos para produção
16. Anexo — SQL / verificações úteis

1. Visão geral da arquitetura

- **Site público** — aplicação **Next.js 14**, alojada na **Vercel** (root directory `site-publico`).
- **Suite de gestão** — aplicação **Vite + React**, alojada na **Vercel** (build `vite build → dist`).
- **Backend** — **Supabase** (PostgreSQL gerido).
- **Auth** — Supabase Auth (email/password), perfis em `public.profiles`.
- **Base de dados** — Postgres com tabelas de domínio (students, staff, programmes, courses, applications, invoices, payments, results, …).
- **Storage** — buckets Supabase para documentos e comprovantes (cartas, EFT).
- **RPCs SECURITY DEFINER** — escritas sensíveis passam por funções controladas (não DML direto).
- **RLS** — Row Level Security ativa nas tabelas sensíveis; cada utilizador só vê o que lhe é permitido.

2. Links e ambientes

Recurso	Endereço
Site público (produção)	https://symanek-site.vercel.app
Suite de gestão (produção)	https://symanek-suite.vercel.app
Vercel — dashboard	https://vercel.com/dashboard (projetos `symanek-suite` e `symanek-site`)
Supabase — dashboard do projeto	https://supabase.com/dashboard/project/zbtxhyxwtemproeomtzu
GitHub — repositório	https://github.com/pedroteixeira201435-source/symanek

> **Nota:** segredos (service role key, DB password, Vercel token) **nunca** devem ser colados neste ou noutro documento. Consultá-los apenas nos painéis Supabase/Vercel ou em variáveis de ambiente do servidor.

3. Variáveis de ambiente

Site público (Next.js — `site-publico`)

Variável	Descrição / valor
`NEXT_PUBLIC_API_MODE`	`supabase` (usa backend real)
`NEXT_PUBLIC_SUPABASE_URL`	URL do projeto Supabase (público)
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Chave anónima (público, protegido por RLS)
`NEXT_PUBLIC_SITE_URL`	URL público do site (para links/cartas)
`SUPABASE_SERVICE_ROLE_KEY`	**Apenas servidor** — usada em rotas API server-side; nunca exposta ao browser

Suite de gestão (Vite)

Variável	Descrição / valor
`VITE_API_MODE`	`http` em produção (lê/escreve via Supabase; `mock` só para demo local)
`VITE_SUPABASE_URL`	URL do projeto Supabase (público)
`VITE_SUPABASE_ANON_KEY`	Chave anónima (público, protegido por RLS)

Nunca expor no cliente / nunca commitar:

- `SUPABASE_SERVICE_ROLE_KEY`

- `SUPABASE_DB_PASSWORD`
- `VERCEL_TOKEN`

Confirmar que a Suite corre com `VITE_API_MODE=http` em produção (não `mock`), caso contrário não persiste dados reais.

4. Como criar acessos reais no Supabase

1. Criar o utilizador em **Supabase Auth** (Authentication → Users → Add user) com `[email do admin]`.
2. Confirmar o email (ou marcar como *email confirmed*).
3. Criar/atualizar o registo correspondente em `public.profiles`.
4. Definir `role` (papel funcional) e `suite_role` (papel na Suite).
5. Para estudante, vincular `students.user_id` ao `auth.users.id` (ver secção 5).
6. Obrigar troca da password inicial, ou definir um processo manual documentado para entregar a `[password temporária]` de forma segura.

Tabela de roles

Role	Âmbito típico
`admin`	Administração geral do sistema
`registrar`	Admissões, matrículas, estudantes
`bursar`	Finanças, faturas, pagamentos
`hr`	Recursos humanos / staff
`teacher`	Docente — cursos e notas atribuídas
`librarian`	Biblioteca
`student`	Portal do estudante (só dados próprios)

`applicant`	Candidato (consulta de candidatura)

5. Como dar acesso a um estudante

1. O estudante tem de existir em `public.students`.
2. Tem de existir um **auth user** associado (Supabase Auth) com `[email do estudante]`.
3. `students.user_id` tem de apontar para `auth.users.id` do mesmo utilizador.
4. O portal só deve mostrar dados do **próprio** estudante (garantido por RLS — validar).
5. Testar o login depois de configurado (ver PDF de UAT, secção I — Student Portal).

> Verificar estudantes sem conta ligada com a query da secção 16 (`students.user_id IS NULL`).

6. Como rever dados antes de produção

Rever cada tabela e confirmar que só existem dados reais (sem resíduo de demo/testes):

Tabela	O que confirmar
`public.students`	Roster real; remover/arquivar demo
`public.programmes`	Programas oficiais
`public.courses`	Cursos reais
`courses.capacity`	Capacidade real preenchida (não 0)
`courses.enrolled`	Inscritos coerentes
`applications`	Candidaturas de teste removidas
`payments`	Sem pagamentos de teste
`invoices`	Faturas coerentes com saldos
`results`	Sem notas de teste publicadas
`profiles`	Todos com `suite_role` definido
`auth.users`	Contas reais; remover contas

demo

> **Alerta:** rever os **~116 estudantes existentes na cloud**. Confirmar se são roster real ou resíduo/demo. **Remover ou arquivar** tudo o que não for real antes do go-live.

7. Capacidades dos cursos

- `capacity = 0` força **lista de espera (waitlist)** — matrículas legítimas caem em espera indevidamente.
- Antes de qualquer matrícula real, **preencher a capacidade real** de cada curso.
- Validar `enrolled` face à capacidade.
- Usar o template/importador do projeto:
  - Template: `supabase/templates/course-capacities.csv`
  - Aplicar: `APPLY_COURSE_CAPACITIES=1 npm run apply:course-capacities`

8. Segurança / RLS / RPC

- Tabelas sensíveis devem ser **RPC-only** (sem DML direto pelo cliente).
- Raw DML bloqueado em: `students`, `staff`, `payments`.
- Todas as escritas passam por RPCs SECURITY DEFINER:

RPC	Ação
`approve_application`	Aprovar candidatura
`reject_application`	Rejeitar candidatura
`mark_paid`	Marcar como pago
`student_upsert`	Criar/atualizar estudante
`student_archive`	Arquivar estudante
`hold_place`	Reservar vaga
`hold_clear`	Libertar reserva
`staff_upsert`	Criar/atualizar staff

`pay_invoice`	Pagar fatura
`confirm_invoice_payment`	Confirmar pagamento de fatura

Confirmar via queries da secção 16 que `authenticated`/`anon` não têm INSERT/UPDATE/DELETE direto nas tabelas sensíveis.

9. Auditoria

- Tabela de rasto: `audit_log`.
- Ações críticas que devem gerar rasto: aprovar candidatura, rejeitar candidatura, pagamento, alterações a estudante, alterações a staff, resultados, documentos.
- Consultar rasto recente com a query da secção 16.

10. CI/CD

- Workflow do GitHub Actions deve estar **ativo** (`.github/workflows/ci.yml`).
- Build da **Suite** deve passar (`npm run build`).
- Build do **site público** deve passar (`cd site-publico && npm run build`).
- Validação de **RLS/RPC deve ser bloqueante** (falha vermelha bloqueia merge).
- `continue-on-error` **não deve existir** nos jobs críticos.
- Fazer push de `.github/workflows/ci.yml` exige um **GitHub PAT com scope `workflow`**.

11. Deploy na Vercel

- **Site público**: root directory `site-publico` (framework Next.js).
- **Suite**: root da app Vite (build `npm run build → dist`, conforme `vercel.json`).
- Confirmar as variáveis de ambiente (secção 3) **antes** do deploy.
- Validar endpoints após o deploy (site abre, Suite faz login, dados reais aparecem).
- **Cuidado com deploy cruzado** — não apontar o projeto do site para o root da Suite (ou vice-versa).

12. Comandos úteis

```
npm run build                          # build da Suite (Vite)

cd site-publico && npm run build       # build do site público (Next.js)



npm run validate:supabase              # valida ligação/estado Supabase

npm run validate:public-site           # valida o site público

npm run validate:uat-core              # valida o núcleo para UAT



npm run cleanup:codex-test-data        # remove dados de teste (Codex)

npm run apply:migration -- <arquivo.sql>            # aplica migração SQL

APPLY_COURSE_CAPACITIES=1 npm run apply:course-capacities   # aplica capacidades
```

13. Rotação de segredos

Ação	Feito?	Notas
Rotacionar o Vercel token usado no handoff		
Rotacionar a DB password do Supabase		
Apagar `.env.codex-handoff`		
Garantir que todos os `.env*` estão no `.gitignore`		
Confirmar que a service role key **não aparece** no histórico Git		

14. Backup e recuperação

- Ativar **PITR** (Point-in-Time Recovery) no Supabase.
- Configurar um **backup externo** (fora do Supabase) com periodicidade definida.
- Fazer um **restore testado** (validar que o backup restaura de facto).

- Documentar data/hora e responsável de cada backup e teste de restore.

15. Critérios técnicos para produção

Item	Estado	Evidência	Responsável	Aprovado?
Builds verdes (Suite + site)				
CI remoto ativo				
RLS/RPC bloqueante				
Dados reais carregados				
Contas reais criadas				
Estudantes revisados (~116)				
Capacidades preenchidas				
UAT aprovado				
Backup / PITR ativo				
Segredos rotacionados				
Demo removido				
Monitoramento ativo				

16. Anexo — SQL / verificações úteis

Executar no SQL Editor do Supabase. Nenhuma query contém segredos.

Políticas RLS nas tabelas sensíveis

```
select tablename, policyname, cmd

from pg_policies

where tablename in ('students','staff','payments');
```

Grants diretos a authenticated / anon (devem ser mínimos)

```
select table_name, grantee, privilege_type

from information_schema.role_table_grants

where table_schema='public'

  and table_name in ('students','staff','payments')

  and grantee in ('authenticated','anon');
```

Contagem de estudantes

```
select count(*) from public.students;
```

Cursos com capacidade 0 (causam waitlist)

```
select code, title, capacity, enrolled

from public.courses

where coalesce(capacity, 0) = 0;
```

Perfis sem suite_role

```
select id, full_name, email, suite_role

from public.profiles

where suite_role is null;
```

Estudantes sem conta de utilizador ligada

```
select id, reference, full_name, email

from public.students

where user_id is null;
```

Candidaturas por estágio

```
select stage, count(*)

from public.applications

group by stage

order by count(*) desc;
```

Rasto de auditoria recente

```
select created_at, actor, action, entity, entity_id

from public.audit_log

order by created_at desc

limit 50;
```